



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA ELÉCTRICA

Informe final - Integración LIDAR 2D para Anticipación de Terreno en Prótesis Transfemoral Activa

PROYECTO DE TÍTULO - IEE3112

Profesor: Felipe Nuñez

Grupo 2

28 DE JUNIO DEL 2026

Índice

1	Hito 1: Propuesta Conceptual y Diseño Funcional	4
1.1	Arquitectura general del sistema	4
1.2	Definición de bloques funcionales	7
1.2.1	Fuente de alimentación	7
1.2.2	Módulo LIDAR 2D	8
1.2.3	Aislación del Módulo LIDAR 2D	8
1.2.4	Plataforma de procesamiento embebido (Raspberry Pi 5)	9
1.2.5	Sistema de comunicación	9
1.2.6	Almacenamiento y monitoreo de datos	11
1.3	Especificación y definición de componentes	13
1.4	Procesamiento y clasificación de datos	14
1.5	Diseño del soporte mecánico	16
1.6	Presupuesto inicial	18
2	Hito 2: Prototipo	21
2.1	Integración mecánica y electrónica del LIDAR 2D en la prótesis	21
2.1.1	Soporte mecánico	21
2.1.2	PCB	22
2.1.3	Consumo estimado	24
2.2	Implementación de sistema	24
2.2.1	LIDAR C1	25
2.2.2	IMU BNO085	25
2.2.3	Red Neuronal	26
2.3	Adquisición de datos	29
2.4	Procesamiento geométrico en tiempo real	31
2.4.1	Filtrado y preprocesamiento de escaneos	32
2.4.2	Detección y estimación de inclinación de rampas y del plano de suelo	32
2.4.3	Detección y estimación de altura de escalones	32
2.4.4	Detección de obstáculos frontales en rango 0.3–3.0 m	33

2.5	Validación de latencia extremo a extremo menor o igual a 200 ms	33
2.6	Métricas de error y matriz de confusión	33
2.7	Validación experimental en entorno controlado con medición de errores geométricos	35
2.8	Repositorio GitHub documentado	36
2.9	Manual técnico de integración, arquitectura y pruebas	36
2.10	Presupuesto final	36
3	Anexo	39

Índice de figuras

Figura 1	Diagrama de Bloques de Alto Nivel	4
Figura 2	Leyenda	4
Figura 3	Diagrama de Bloques de Bajo Nivel (Completo)	5
Figura 4	Diagrama de Bloques de Bajo Nivel (Parte 1)	5
Figura 5	Diagrama de Bloques de Bajo Nivel (Parte 2)	6
Figura 6	Batería para validación experimental	8
Figura 7	Diferentes topologías ML	15
Figura 8	Representación de la pierna y su modelo asociado.	16
Figura 9	Curvatura de la pierna.	17
Figura 10	Soporte mecánico - Diseño 3D.	17
Figura 11	Soporte mecánico - Aplicación Física.	18
Figura 12	Soporte mecánico diseñado para la prótesis.	21
Figura 13	Solapas del soporte mecánico diseñado para la prótesis	22
Figura 14	Diseño de Placas en Software EasyEDA.	23
Figura 15	Ambas PCB montadas una sobre otra en aplicación final	24
Figura 16	Diagrama red iteración 1	28
Figura 17	Diagrama red iteración 2	29
Figura 18	Pagina Local para la Toma de Muestras	30
Figura 19	Imágenes de Ejemplo de Adquisición de Datos	31

Figura 20	Latencia	33
Figura 21	Matriz de confusión sobre Test-dataset para datos binarios	34

Índice de tablas

Tabla 1	Especificaciones de componentes para integración de LIDAR en prótesis transfemoral activa	13
Tabla 2	Tabla con principales consumos	14
Tabla 3	Presupuesto Hito 1	20
Tabla 4	Tabla con principales consumos	24
Tabla 5	Etiquetado.	27
Tabla 6	Métricas de error para 100 épocas de entrenamiento	33
Tabla 7	Presupuesto Hito 2.	38

1. Hito 1: Propuesta Conceptual y Diseño Funcional

1.1. Arquitectura general del sistema

La siguiente figura muestra el diagrama de bloques de alto nivel, el cuál entrega una primera idea simplificada de cuál es el flujo del sistema en su totalidad. A modo de explicación, el sistema inicia con un bloque de sensores que miden el entorno, los cuáles entregan medidas a la Raspberry Pi 5. Esta se encarga del procesamiento de las medidas, la predicción mediante uso de algoritmos de Machine Learning y la posterior comunicación para validación del usuario monitor. El objetivo es que las predicciones sirvan para ejecutar una reacción por parte de la prótesis, aunque se escapa de los objetivos de este proyecto. Finalmente, el sistema posee su sub-sistema de alimentación, que le otorga independencia.

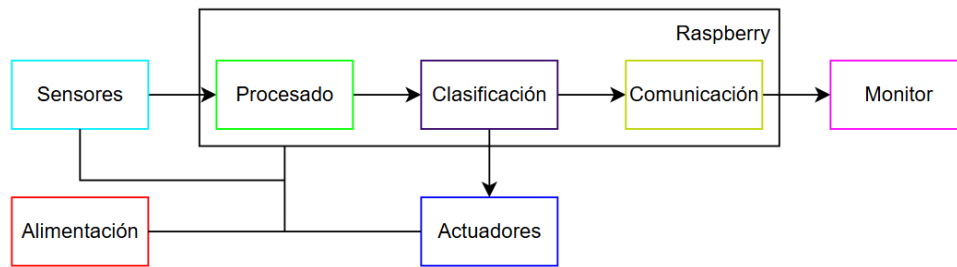


Figura 1: Diagrama de Bloques de Alto Nivel

Como se puede observar en la figura, los bloques siguen cierto código de colores, el cuál se detalla en la leyenda a continuación.



Figura 2: Leyenda

Una vez entendido el sistema de manera general, se puede proceder a analizar el diagrama de bloques de bajo nivel. El cual, es escala macro, se ve como sigue. Luego, se detallará cada sub-sistema con detenimiento.

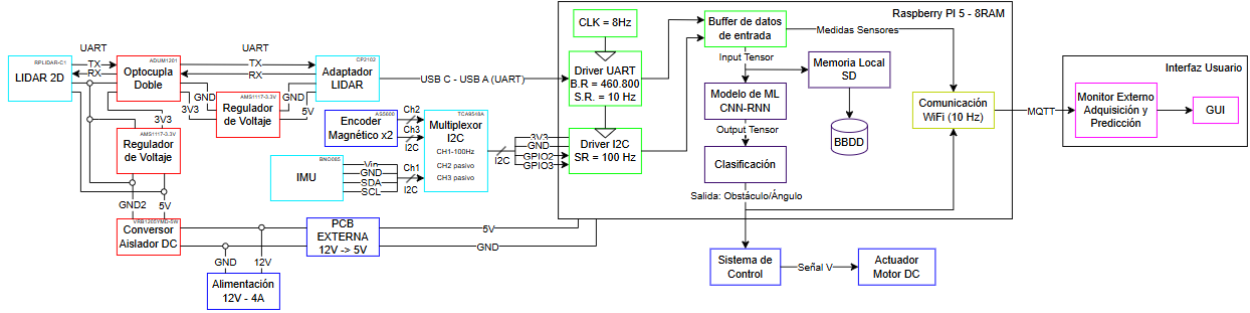


Figura 3: Diagrama de Bloques de Bajo Nivel (Completo)

Por motivos de la baja resolución de este en el informe, se procede a analizar el diagrama por partes.

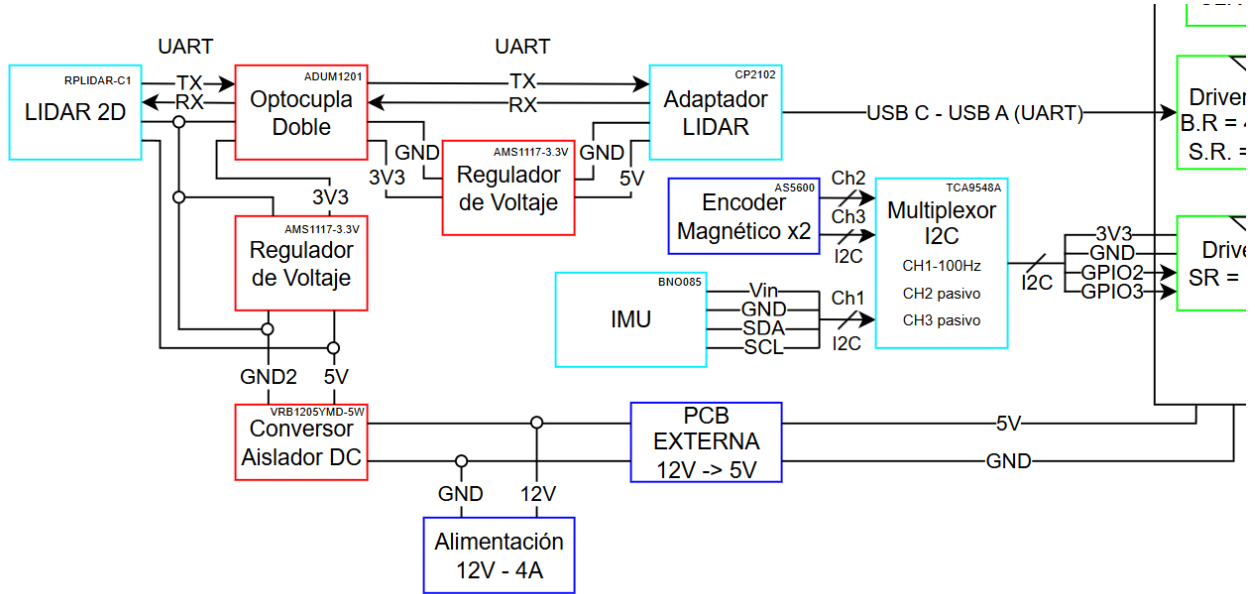


Figura 4: Diagrama de Bloques de Bajo Nivel (Parte 1)

La primera parte de este diagrama, incluye los bloques de sensores en color celeste, alimentación y considera elementos externos. Esta parte es lo que concentra la mayor parte del Hardware del sistema. En cuanto a los sensores, se aprecian el sensor principal: el LIDAR,

la unidad de medición inercial (IMU) y sus adaptadores. Lo relacionado al LIDAR funciona mediante protocolo UART, mientras que lo relacionado al IMU usa protocolo I2C. Por otra parte, en color rojo, se encuentra lo relacionado a la alimentación. Estos bloques aparecen principalmente para alimentar correctamente, y sobre todo proteger el sensor LIDAR y aislarlo contra posibles amenazas, por su sensibilidad y costo. Se explicará más en detalle en las secciones correspondientes. Finalmente, en color azul, se encuentran elementos externos que ya forman parte del proyecto, pero que decidimos tomar en consideración, para una correcta cohesión con el desarrollo del proyecto. Estas son la alimentación de las baterías actuales, la PCB del sistema de baterías y los *encoders* magnéticos.

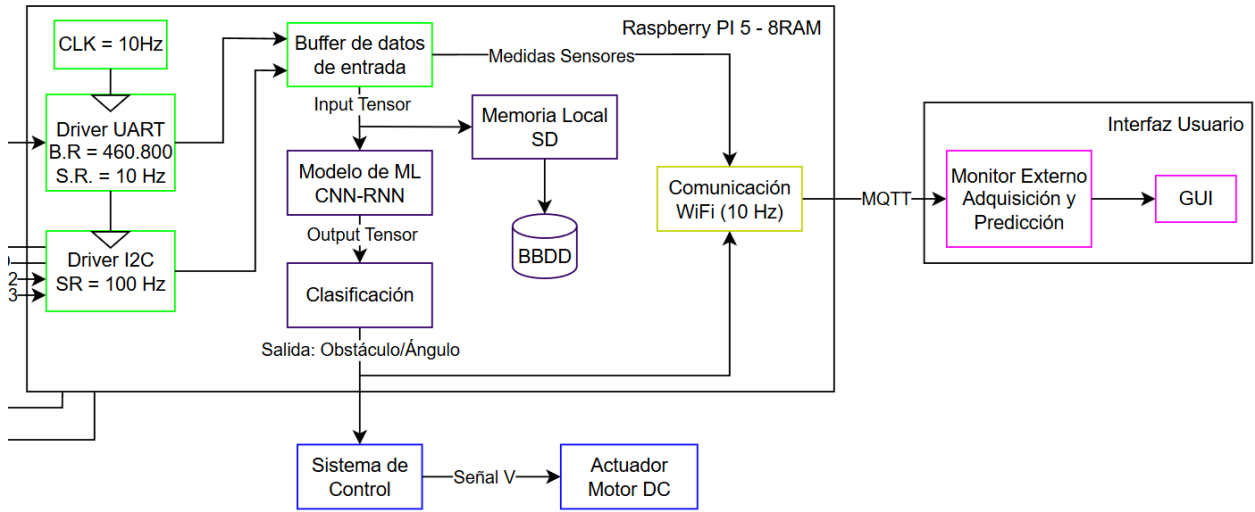


Figura 5: Diagrama de Bloques de Bajo Nivel (Parte 2)

La segunda parte de este diagrama incluye los sub-sistemas de procesamiento, predicción, comunicación y monitoreo de usuario. La primera parte, en color verde, muestra la coordinación de los 2 sistemas de adquisición de datos mediante los sensores, para formar los datos de entrada para el modelo de red neuronal. Luego, en color morado, se encuentra lo relacionado al algoritmo de predicción como tal, lo que incluye también la toma y guardado de datos de forma local, para usarlos a lo largo del entrenamiento del modelo. En color amarillo, se incluye el bloque de comunicación, que servirá durante las implementaciones finales y *debugueo* correspondiente. Al final de la cadena, en rosado, está el bloque que corresponde al monitoreo del sistema de forma remota, mediante una interfaz para el usuario. Por último,

en la imagen se aprecian 2 bloques de color azul, que nuevamente son funciones externas al objetivo del proyecto, relacionadas al sistema de reacción/actuación frente a la predicción.

1.2. Definición de bloques funcionales

1.2.1. Fuente de alimentación

La prótesis transfemoral activa ya cuenta con una batería integrada que alimenta la electrónica de control existente. Esta misma fuente de energía será utilizada para suministrar potencia a los nuevos subsistemas de percepción (LIDAR e IMU).

Dado que la batería ya se encuentra dimensionada y validada para la operación de la prótesis, no es responsabilidad de este proyecto redefinir, reemplazar o caracterizar la fuente de alimentación. En su lugar, nos enfocamos en asegurar que los nuevos componentes (RPLIDAR C1 e IMU BNO085) sean compatibles con los niveles de voltaje y corriente disponibles en el sistema existente.

La integración se realiza mediante un convertidor DC-DC aislado modelo VRB1205YMD de 12 a 5 V de 5 W, que permite adaptar la tensión de la batería a los requerimientos específicos del LIDAR (4.8-5.2V a 260 mA típico) y para poder suministrar la corriente peak del Lidar (1A en la partida). Este convertidor actúa como interfaz entre la fuente centralizada de la prótesis y los nuevos sensores, protegiendo ambos subsistemas de variaciones en la tensión de la batería. Mientras que la IMU (3-5V a 3-5 mA) se alimentará desde la misma fuente utilizada por la Raspberry 5 Pi. Para las fases de prototipado y validación experimental, se utilizará una batería portátil de 5000 mAh que proporciona conexión USB estándar y voltaje de salida 5V.



Figura 6: Batería para validación experimental

1.2.2. Módulo LIDAR 2D

Para este proyecto se utilizará el modelo RPLIDAR C1, el cual tiene un rango de medición de 12 metros para objetos claros y de 6 para oscuros. Además, presenta una frecuencia de muestreo de 5 kHz, un campo de visión 360° y una precisión de ± 30 mm.

Utiliza un puerto TTL UART con un baud rate de 460,800 bps para proporcionar información de distancia (mm), ángulo (grados), señal de inicio de escaneo y datos de reflectividad (2.5D). Los requerimientos de funcionamiento son principalmente una alimentación de 5 V y 230 mA.

El dispositivo es ligero, tiene un peso aproximado de 110 g, y es resistente a polvo y salpicaduras. Sin embargo, se debe tener la precaución de no exponer el láser directamente al sol puesto que podría dañar irreversiblemente su funcionamiento.

1.2.3. Aislación del Módulo LIDAR 2D

Un LiDAR 2D es un sensor electromecánico de operación continua. Por esta razón, compartir la misma referencia de tierra con una Raspberry Pi 5 puede ocasionar diversos inconvenientes. El primero de ellos es la inyección de ruido electromecánico. Los motores, al ser cargas inductivas, junto con su conmutación electrónica, generan picos de voltaje que se inyectan

directamente en el plano de tierra. Otro problema importante son las corrientes de irrupción (inrush), que consisten en corrientes transitorias de arranque capaces de provocar caídas momentáneas de tensión, comprometiendo así la estabilidad de los rieles de alimentación de la Raspberry Pi 5.

La solución a estos problemas es la aislación galvánica del motor con el sistema de control, tanto para proteger el LIDAR 2D como la Raspberry Pi 5. Tal como se mencionó en etapas anteriores, se utiliza un conversor DC-DC tipo Buck conectado en el lado de alto voltaje a 12V desde la batería (ya integrada en la prótesis) y en la lado de bajo voltaje a 5V conectado a los rieles de alimentación del LIDAR 2D.

Para mantener el aislamiento galvánico en las líneas de comunicación entre la Raspberry Pi 5 y el LiDAR 2D, se implementará el aislador digital ADuM1201. Dado que los niveles lógicos de salida de este módulo dependen exclusivamente de las tensiones aplicadas a sus rieles de alimentación, y considerando que tanto el LiDAR 2D como el adaptador USB operan con lógica de 3.3 V, resulta importante el uso de dos reguladores de tensión AMS1117-3.3 para fijar dicho voltaje en ambos extremos del circuito.

1.2.4. Plataforma de procesamiento embebido (Raspberry Pi 5)

La Raspberry Pi 5 es un pequeño computador de bajo costo con un rendimiento sorprendente para su tamaño. Su procesador cuádruple núcleo a 2.4 GHz la hace entre 2 y 3 veces más rápida que la versión anterior, y su chip gráfico alcanza cerca de 0.077 TFLOPS, suficiente para tareas de ML. Incluye WiFi de doble banda y Bluetooth 5.0, lo que facilita su conexión inalámbrica en cualquier entorno. También estrena una ranura PCIe que permite conectar discos SSD de alta velocidad, algo nunca visto en versiones anteriores. Con hasta 16 GB de memoria RAM y un consumo eléctrico muy bajo, es ideal para proyectos embebidos, automatización y computación en el borde de la red.

1.2.5. Sistema de comunicación

Los bloques funcionales presentados previamente necesitan transmitir datos entre ellos para un correcto funcionamiento del sistema. En general los distintos bloques comunican o reciben

datos desde la Raspberry, que actúa en este caso como nodo central del sistema. A continuación se enumeran las principales conexiones para la transmisión de datos y los protocolos utilizados.

- Conexión LiDAR → Raspberry:

La conexión entre estos dos nodos se realiza mediante protocolo UART con una conexión micro-usb. Lo anterior se justifica debido a las especificaciones del LiDAR, ya que transmite por defecto sus datos con protocolo UART. Además, el adaptador usb del LiDAR facilita la conexión directa con la Raspberry a través de uno de sus puertos.

El formato en que se envían los datos será la distancia de la medición del láser del LiDAR y el ángulo de este.

- Conexión IMU → Raspberry El sensor IMU se conecta con la Raspberry a través de protocolo I2C. Se eligió este protocolo debido a que es el que tiene el sensor por defecto, además de ser simple y robusto de implementar para esta aplicación.

Un aspecto relevante a tener en cuenta es que el sensor no se conectará directo a terminales I2C de la Raspberry, debido a que estos están siendo utilizados por otros sensores que son parte de la implementación de la prótesis, pero externos a este proyecto. Debido a lo anterior, se utilizará un multiplexor I2C al que se conectará otro sensor además del IMU, de tal forma que puedan compartir un solo puerto I2C de la raspberry.

El multiplexor a utilizar se conectará a los 3 sensores y a un solo puerto de la raspberry. Cómo se pudo observar en el diagrama de bloques inicial, los tres sensores serían dos encoders magnéticos y el IMU, la estrategia para que coincidan los tiempos del LiDAR y el IMU es utilizar el Lidar a 10 Hz y el IMU a 100 Hz una vuelta completa (ambos se pueden utilizar en estas configuraciones de muestreo).

De esta forma, el multiplexor escuchará principalmente al IMU enviando mediciones cada 100 Hz. Luego, cada 10 Hz, se enviarán las muestras del encoder para que coincidan con las mediciones del LiDAR, coordinandose de esta forma la utilización del puerto I2C, las mediciones del IMU y la medición del LiDAR con sus respectivas frecuencias de muestreo.

Los datos que se recibirán del IMU corresponden a la posición relativa del LiDAR respecto a cierto marco de referencia. Esta medición se envía mediante cuaterniones, dado que el dispositivo utilizado tiene la capacidad de enviar sus datos directamente en este formato.

- **Conexión Raspberry ↔ Interfaz de monitoreo:**

El objetivo de esta conexión es enviar los datos de medición del LiDAR y el IMU, además de las clasificaciones realizadas por el modelo de Machine Learning a una plataforma externa donde se pueda monitorear el funcionamiento de la prótesis.

Para esta funcionalidad se transmitirán los datos vía WiFi con protocolo MQTT. La razón de esto es que se necesita una transmisión inalámbrica de los datos, por lo que se decidió utilizar el módulo WiFi que tiene integrado la Raspberry.

Por otro lado, se eligió el protocolo MQTT debido a las características de la aplicación, este protocolo está pensado para dispositivos IoT donde el envío de datos se realiza en condiciones de redes inestables y anchos de banda limitados, además permite una lectura simple de los datos desde otros dispositivos como un ordenador o una página web para mostrar los datos recibidos en tiempo real.

Los datos se enviarán cada vez que el modelo realice una clasificación, el formato de envío de los datos será la clasificación con el tipo de obstáculo y características como ángulo de inclinación en caso de una rampa y altura del escalón en caso de una escalera. Sumado a lo anterior, se enviará en el mismo paquete las entradas del LiDAR y el IMU con las que se realizó la clasificación.

1.2.6. Almacenamiento y monitoreo de datos

Como se observa en el diagrama de bloques de bajo nivel del sistema, los datos tienen dos destinos, un almacenamiento en la Raspberry y un envío remoto mediante WiFi y MQTT que fue adelantado en la sección anterior, y hacia el bloque controlador de la prótesis. En esta sección se explicarán más a detalle ambos destinos de los datos luego de la clasificación.

- Almacenamiento local: Su objetivo es almacenar únicamente las mediciones del Lidar y el Imu para posteriormente seguir entrenando al modelo con estos datos. El almacenamiento se realiza en una base de datos local dentro de la raspberry en su tarjeta SD, para posteriormente cargarlos en otro dispositivo y trabajar con ellos.
- Envío remoto para monitoreo: Este flujo de datos es el que se explicó en la sección anterior. El objetivo es que cada cierto envíe las clasificaciones realizadas y los datos de entrada con los que realizó la clasificación.

Para este proyecto, estos datos serán leídos desde el Broker MQTT desde un computador que generará una interfaz gráfica local con un script en Python. El objetivo de esto es mostrar de una forma amigable y útil para trabajar los datos en los experimentos para testear el funcionamiento de la implementación del proyecto.

Cabe destacar que si bien la implementación de la interfaz gráfica mediante la lectura de datos desde un computador, gracias a la arquitectura MQTT es posible también escalar hacia una página web donde se lean los datos en tiempo real, sin embargo, esto escapa del proyecto y se puede implementar en una etapa más avanzada de la prótesis.

- Controlador: Finalmente, el controlador de la prótesis que envía la acción de control al motor actuador, recibe únicamente la clasificación de obstáculos realizada por el modelo, donde recibe el tipo de obstáculos y datos relevantes como la altura del escalón si el obstáculo es una escalera, el ángulo de inclinación si es una rampa, entre otros. Este bloque aún no está implementado y no forma parte del proyecto actual, por lo que solo se define cómo recibirá los datos.

1.3. Especificación y definición de componentes

Componente	Modelo/Especificación	Parámetros Clave
Sensor LIDAR 2D	RPLIDAR C1 (C1M1-R2)	Alcance: 0.05–12 m Frecuencia: 10 Hz @ 600 RPM Resolución: 0.72° Precisión: ± 30 mm Peso: 110 g
IMU 9-DOF	Adafruit BNO085	Acelerómetro: ± 16 g Giroscopio: $\pm 2000^\circ/\text{s}$ Magnetómetro: $\pm 4900 \mu\text{T}$ Interfaz: I ² C/UART/SPI Fusión sensorial: Cortex-M0 Voltaje entrada: 3–5 V DC Consumo: $\sim 3\text{--}5$ mA típico Precisión angular: $\pm 2^\circ$
Interfaz LIDAR	UART TTL 3.3V XH2.54-5P	Velocidad: 460800 bps Modo: 8n1 Tensión salida: 0–3.5 V
Procesador	Raspberry Pi 5 Model B	CPU: BCM2712 Quad-Core ARM Cortex-A76 Frecuencia: 2.40 GHz Arquitectura: 64-bit aarch64 RAM: 8 GB GPU: Broadcom VideoCore VI Sistema operativo: Debian GNU/Linux 13 (Trixie) Kernel: Linux 6.12.75+rpt-rpi-2712 Compatible con ROS 2

Tabla 1: Especificaciones de componentes para integración de LIDAR en prótesis transfemoral activa

A continuación se presenta una tabla con los consumos energéticos principales de los componentes de la implementación. El resto de componentes de este proyecto no se tomaron en cuenta debido a que sus consumos son despreciables en relación a los que se presentarán a continuación:

Esta tabla permite calcular la autonomía de la batería del sistema implementado en este proyecto, así como el de la prótesis en su totalidad al considerar los consumos del resto de

Componente	Consumo (W)
Sensor LiDAR 2D	1.35 W
Conversor DC-DC Aislado (pérdida por conversión)	0.25 W
Raspberry PI	10 W

Tabla 2: Tabla con principales consumos

bloques ya implementados en la prótesis.

Considerando que el power bank utilizado para las pruebas tiene una capacidad de 5000 mAh, la autonomía de la batería es de aproximadamente 2 horas y 10 minutos.

1.4. Procesamiento y clasificación de datos

Existen diversas topologías enfocadas en nubes de puntos para detectar objetos como PointNet o PointNet++ las cuales usan simplemente MLP. Ahora, estas topologías simples no aprovechan el hecho de que el Lidar entrega los datos en forma ordenada $r(\theta)$ por lo que los modelos propuestos usan CONV1D para aprovechar estas ventanas de datos, los cuales al ser cercanos proporcionan cierta información que puede ser captada por un kernel de convolución. Por otro lado no se usa CONV2D porque sería entregar la nube de puntos como una imagen, lo cual aumenta el tiempo de procesamiento.

En base a esto, los papers siempre usan como capa base las convolucionales 1D, adoptando diversas cantidades de neuronas, max pooling, agregando más información al input como pueden ser los datos de un IMU, o agregando capas con memoria como RNN, GRU o STM. Todo depende de la dificultad que se quiere conseguir como output de la red.

A continuación se presentan dos modelos, los cuales tienen diversos fines y grados de dificultad

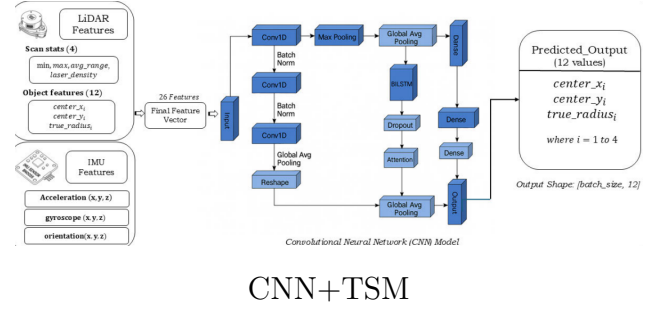
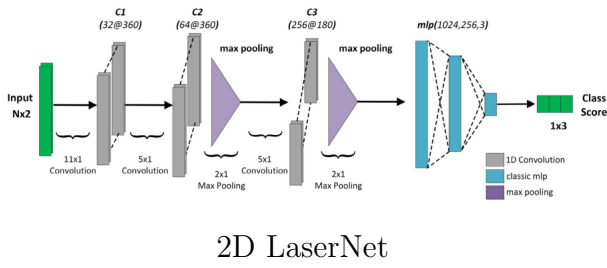


Figura 7: Diferentes topologías ML

La primera imagen es una red muy simple, la cual transforma los datos $r(\theta) \rightarrow (x, y)$ de ahí el input $N \times 2$, cada uno de ellos se procesa con capas CONV1D por rapas separadas, aplican max pooling para reducir dimensionalidad quedándose solo con datos importantes, finalmente condensan todo y se procesa por la típica MLP. Esta red extrae diversas clases, y fue creada para la detección de elementos como corredores, pasillos, piezas, puertas, entre otras. Se consigue un accuracy superior al 95 % con excepción de las puertas. La detección ocurre en tiempo real y puede ser usada sobre robots para el monitoreo del entorno.

La segunda imagen implementa los datos del Lidar en conjunto con el IMU, acá los datos se procesan para conseguir distintos parámetros, los cuales se suman a una estimación de los objetos cercanos y los datos del IMU dando vector de 26 entradas como input. La idea es estimar la posición (x,y) y el radio de un objeto. Esta red es muy precisa e implementa una capa STM para agregar memoria al sistema, de esta forma la red sabe de alguna manera que el objeto o el robot pueden estar en movimiento y lo considera debido a que su salida no es una simple clasificación, si no que la dimensionalidad y la posición también.

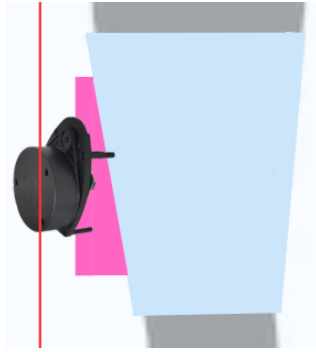
El modelo propuesto considera el punto medio entre ambos puntos, considerando capas convolucionales 1D como base y probar métodos como:

- Input coordenadas $r(\theta)$ con información de IMU
- Input coordenadas (x, y) con información de IMU considerando procesamiento por capas separadas

- Probar distintas cantidades de capas, neuronas, agregar GRU o RNN y generar un trade off entre tiempo de procesamiento, uso de la CPU y accuracy, para saber si vale la pena en este contexto agregar memoria a la red o no.

1.5. Diseño del soporte mecánico

Para el soporte mecánico se diseñó una rampa que pudiera corregir la inclinación de la pierna y de esta forma se orientara el laser de forma tal que este perpendicular al suelo. Para calcular el ángulo de inclinación se utilizó una foto de referencia, se dibujó un triángulo rectángulo y se midió digitalmente los largos de sus catetos. Posteriormente, se utilizaron dichos valores y se obtuvo un ángulo de 6.12° .



Dibujo esquemático de la pierna.



Cálculo de la pendiente.

Figura 8: Representación de la pierna y su modelo asociado.

También fue necesario calcular la curvatura natural de la pierna para que el soporte calce de la manera más ergonómica posible. Para esto se utilizó papel aluminio y se moldeó la forma de la pierna a la altura donde se ubicará posteriormente el LiDAR. El radio del círculo que define esa curvatura es 166.73 mm.

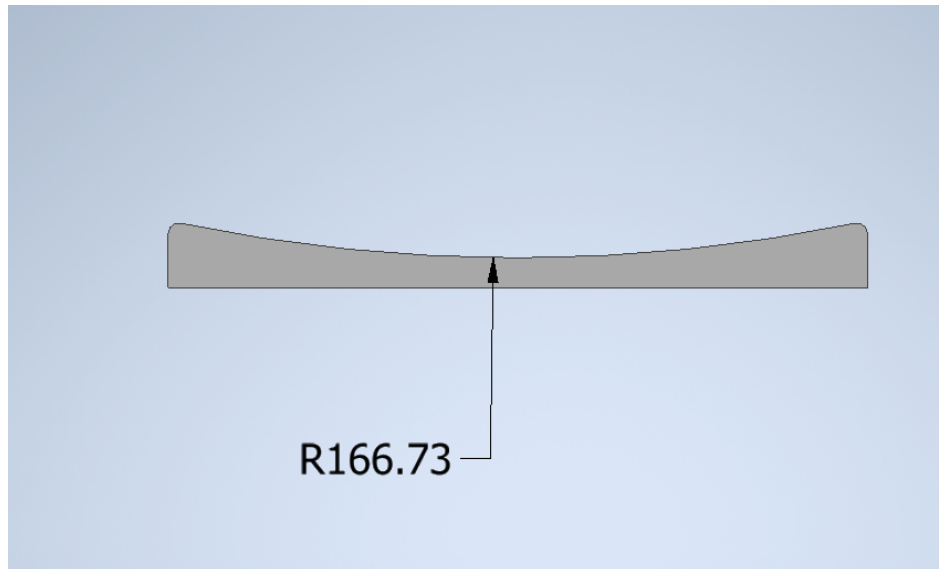
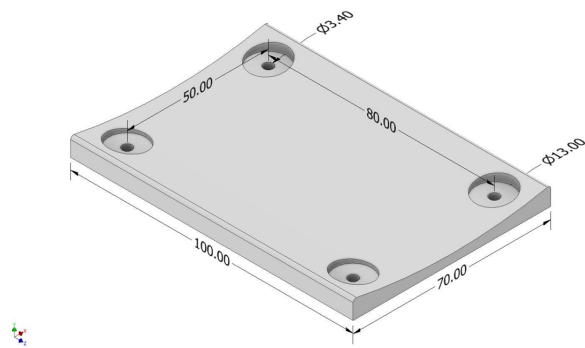
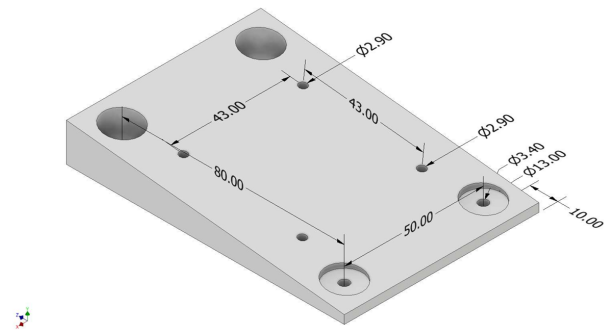


Figura 9: Curvatura de la pierna.

A continuación se adjuntan los modelos diseñados.



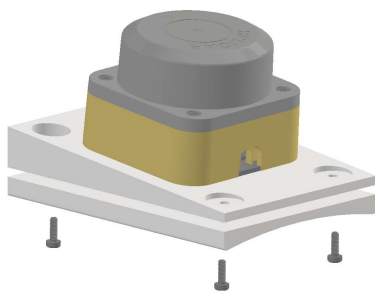
Base curvada para soporte a la pierna.



Rampa de nivelación para el láser.

Figura 10: Soporte mecánico - Diseño 3D.

Finalmente, se adjunta la pieza renderizada, impresa y montada.



Piezas renderizadas



Montaje de Pruebas en Físico

Figura 11: Soporte mecánico - Aplicación Física.

1.6. Presupuesto inicial

Para el desarrollo de este proyecto se estima que se necesita contemplar el presupuesto, expuesto en la tabla 3, para construir el prototipo inicial.

Descripción	Precio	Comentarios/Justificación
Banda de compresión musculera	\$11.100	Soporte para pruebas: en esta banda se montará el sistema del LiDAR para realizar pruebas previas a la propuesta de instalación del sensor en la prótesis.
Filamento Impresión 3D PLA 1.75 mm 2 kg	\$26.000	Soporte para pruebas: se necesita imprimir el soporte donde montaremos el LiDAR y demás componentes del sistema junto al agarre a la banda.
Kit 830 Puntos de Conexión Componentes Electrónicos	\$10.760	Soporte para pruebas: se necesitan componentes varios para hacer pruebas y prototipos.

Continúa en la siguiente página...

Tabla 3 – Continuación de la página anterior

Descripción	Precio	Comentarios/Justificación
Tornillos M2.5 8 mm (100 uds)	\$6.581	Soporte para pruebas: para unir las dos partes del prototipo 3D que agarran a la banda.
Velcro Autoadhesivo	\$7.480	Soporte para pruebas: se utilizará para poder remover y ubicar fácilmente la batería en la banda utilizada. Se pegará una banda de velcro a la batería que usaremos para alimentar el sistema con el objetivo de que sea más simple cargarla.
Batería de alimentación	\$11.790	Soporte para pruebas: se utilizará para alimentar el sistema (5000 mAh).
IMU BNO085	\$28.678	Implementación: se utilizará para determinar el ángulo de inclinación de la pierna en tiempo real. Comentario: no se encontró un proveedor local confiable y a un precio razonable.
Conversor 12V-5V	\$4.205	Alimentación aislada LIDAR 2D.
Raspberry Pi 5 shield	\$6.280	Solicitado por la contraparte para mayor modulación en la aplicación.
Nivel Digital	\$26.432	Permite aislar la línea de comunicación entre LIDAR 2D y Raspberry Pi 5
Aislador Digital ADUM1201	\$2.158	Para realizar mediciones confiables y precisas de niveles de inclinación para validar las pruebas y resultados.

Continúa en la siguiente página...

Tabla 3 – Continuación de la página anterior

Descripción	Precio	Comentarios/Justificación
Módulo Regulador de Voltaje 3.3V AMS1117	\$2.456	Permite reducir voltaje de 5V a 3.3V
Multiplexor I2C de 8 Canales TCA9548A	\$2.983	Controlar el uso de Comunicación I2C
Total	\$146.903	

Tabla 3: Presupuesto Hito 1

2. Hito 2: Prototipo

2.1. Integración mecánica y electrónica del LIDAR 2D en la prótesis

2.1.1. Soporte mecánico

Se diseñó una pieza 3D para sostener el LiDAR y las PCBs en una misma posición independiente del movimiento de la prótesis, esta ubicación se presenta a continuación.

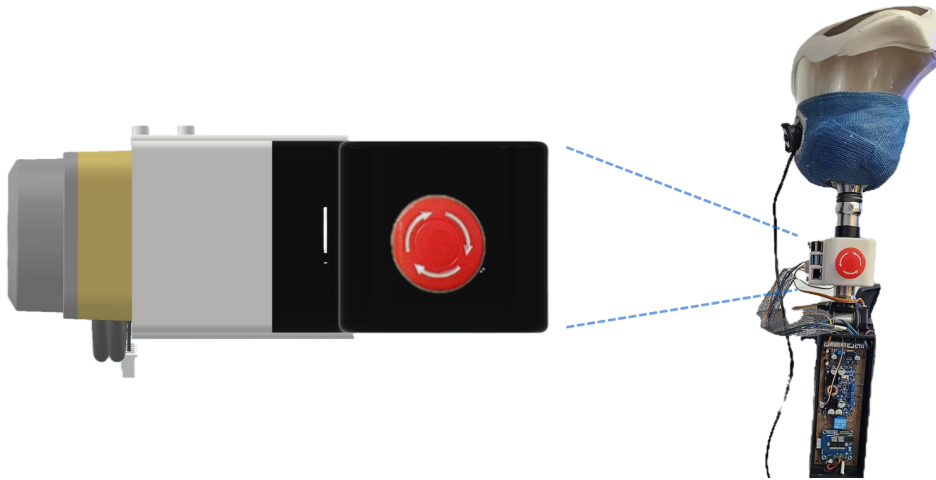


Figura 12: Soporte mecánico diseñado para la prótesis.

La pieza diseñada es una caja que en su interior contiene las PCBs previamente fijadas con los espaciadores, mientras que en su exterior tiene fijado con pernos el LiDAR. Además, para fijar esta pieza a la prótesis se diseñaron unas solapas, como se muestra en la figura 13, en donde se ubicarán unos tornillos de soporte.

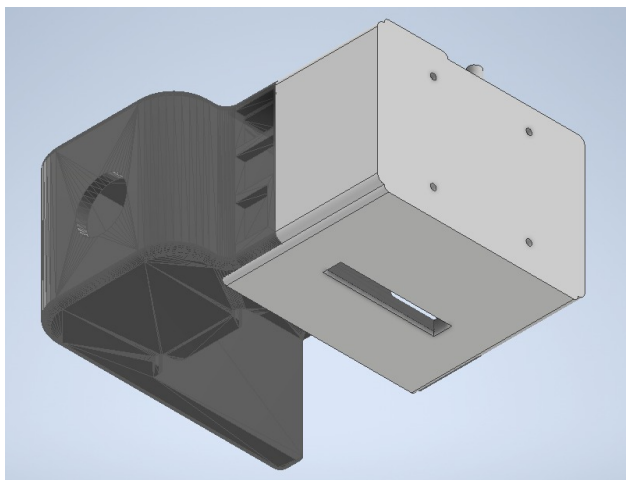
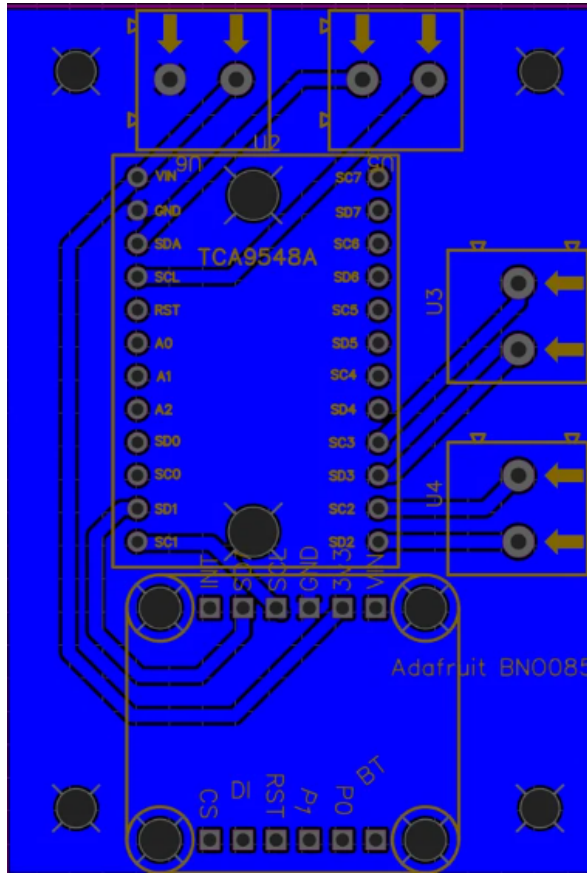


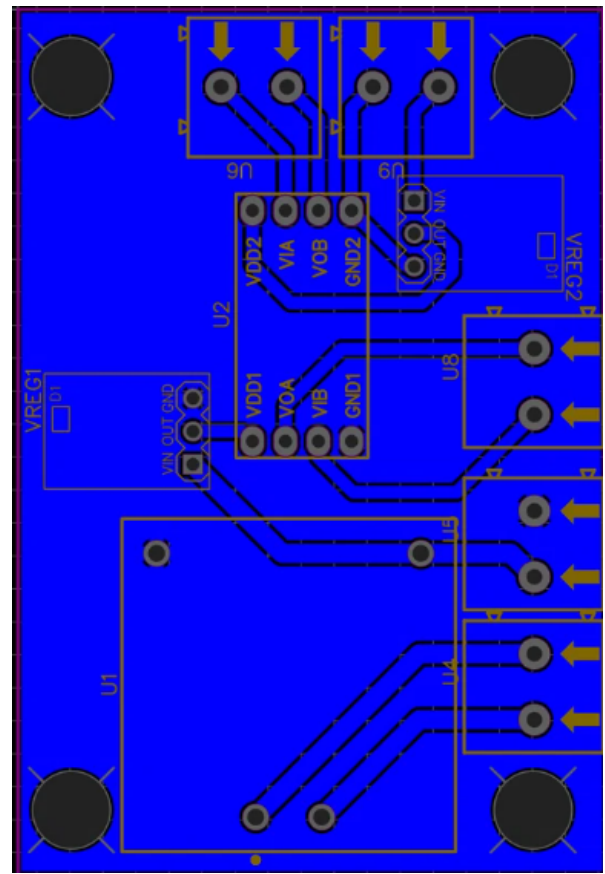
Figura 13: Solapas del soporte mecánico diseñado para la prótesis

2.1.2. PCB

Se desarrollaron las siguientes dos placas para suplir las funciones de protecciones eléctricas, coordinación de sensores e incorporación apropiada al prototipo ya diseñado por la contraparte. Estas se exponen y detallan a continuación.



(a) PCB superior - IMU y MUXI2C



(b) PCB inferior - Reguladores y Protecciones

Figura 14: Diseño de Placas en Software EasyEDA.

La primera placa, se organiza en torno al multiplexor de bus I²C TCA9548A. Como las unidades inerciales BNO085 comparten una misma dirección I²C fija y no pueden coexistir en un mismo bus, el multiplexor habilita un canal a la vez y permite leerlas de forma secuencial sin conflictos. Así se obtienen varias referencias de orientación del segmento de la prótesis, que el sistema utiliza para discriminar entre escaleras, rampas y obstáculos. La alimentación y el bus I²C llegan a esta placa ya aislados desde la primera, de modo que el aislamiento galvánico se mantiene a lo largo de todo el trayecto.

La segunda placa, toma los 12 V de la batería y los reduce a 5 V mediante un convertor Buck, generando dos rieles independientes a ambos lados de la barrera de aislamiento. El integrado U2 aísla las líneas de comunicación entre ambos dominios, separa el lado primario (VDD1/GND1, ligado a la batería) del lado secundario (VDD2/GND2, ligado a los sensores), de modo que la electrónica de adquisición no comparta tierra con el lado de potencia.

Ambas placas fueron diseñadas para montarse la una sobre la otra, con tal de aprovechar al máximo el espacio y, sobre todo, adaptarse al prototipo ya implementado por la contraparte en la prótesis. Estas se unieron mediante elementos de tornillería para aplicaciones de PCB. En la imagen se pueden observar ambas PCB y su distribución antes de ser incorporadas en la carcasa diseñada.



Figura 15: Ambas PCB montadas una sobre otra en aplicación final

2.1.3. Consumo estimado

Componente	Consumo (W)
Sensor LiDAR 2D	1.35
Convertor DC-DC Aislado (pérdida por conversión)	0.25
Raspberry PI	10
Multiplexor TCA9548A	0.7
Regulador de voltaje AMS1117 5V A 3.3 v (x2)	0.06
Aislador UART ADUM1201	0.0264

Tabla 4: Tabla con principales consumos

2.2. Implementación de sistema

El código implementado para el funcionamiento del sistema, se realizó mediante el lenguaje de programación Python. Este consta de tres *scripts*, el principal denominado "script_red.IMU_LI-DAR.py", contiene la base para el funcionamiento de los sensores IMU y LIDAR, en conjunto con el procesamiento de la información y la inferencia de la red neuronal en tiempo real.

Para asegurar el cumplimiento de la condición de latencia, se decidió optar por la utilización de dos de los cuatro núcleos disponibles de la Raspberry Pi 5. Para ello se utilizó funciones de la librería multiprocessing.

Uno de los núcleos, se utiliza exclusivamente para la adquisición de la información, tanto de la IMU como el LIDAR, mientras que el segundo, se utiliza para el filtrado y procesamiento de los datos, además de la inferencia en tiempo real de la red neuronal diseñada.

2.2.1. LIDAR C1

El LIDAR C1 se conecta a la Raspberry Pi 5 mediante puerto serie USB y opera de forma continua. Cada vuelta produce un conjunto de puntos con ángulo y distancia medida, que varía en cantidad según la velocidad de giro y la tasa de muestreo del sensor. Al completarse cada vuelta, los puntos se ordenan según su ángulo y se interpolan a un vector fijo de 450 entradas, correspondientes a ángulos uniformemente espaciados cada $0,8^\circ$ entre 0° y $359,2^\circ$, donde los valores de las entradas corresponden a las distancias medidas por el LiDAR. Los ángulos sin distancia asignada o con falta de angulos superiores a 5° se rellenan con la distancia máxima del sensor (12.000 mm o 12 m).

Antes de la interpolación de los datos, el escaneo del LiDAR se corrige geométricamente usando el vector de gravedad entregado por el IMU BNO085. Esta corrección rota el eje de referencia del escaneo para compensar la inclinación actual de la prótesis, de modo que el plano del suelo quede siempre en la misma posición angular relativa independientemente de la postura del dispositivo.

El vector de 450 distancias nivelado y normalizado es la entrada a la red neuronal, que produce como salida tanto clasificaciones binarias de obstáculos como las respectivas distancias o ángulos si aplica el caso.

2.2.2. IMU BNO085

El sensor BNO085 se comunica con la Raspberry mediante el bus I²C. Se utilizan tres mediciones del sensor: acelerómetro (`BNO_REPORT_ACCELEROMETER` en código), giróscopio (`BNO_REPORT_GYROSCOPE` en código) y vector de gravedad (`BNO_REPORT_GRAVITY` en código). Este último es estimado por el propio BNO085 mediante su algoritmo de filtro de Madgwick, el cual filtra las componentes dinámicas de la aceleración para entregar únicamente la componente gravitacional estática.

La lectura del sensor se ejecuta en un hilo secundario dentro del proceso de adquisición de da-

tos, corriendo a una frecuencia de 100 Hz. En cada iteración se leen las tres tuplas (x, y, z) del acelerómetro, giróscopio y vector de gravedad.

Para suavizar el ruido en las lecturas del vector de gravedad, se aplica un filtro de promedio móvil con una ventana de 10 muestras, independiente para cada eje del vector de gravedad.

Los valores filtrados se almacenan en un diccionario global compartido, junto con las lecturas crudas del acelerómetro y del giróscopio. Este diccionario es utilizado por el hilo principal al completar cada vuelta del LiDAR para luego realizar la corrección de las mediciones.

2.2.3. Red Neuronal

El modelo de *machine learning* fue desarrollado para lograr estimar características espaciales del entorno, como:

- Obstáculos frontales/traseros
- Rampas con su respectiva inclinación
- Escaleras con la respectiva altura del escalón

Para generar estas etiquetas, durante la adquisición de datos (la cual se ahonda más adelante en [2.3]), se asignaron tanto etiquetas discretas, para la detección de obstáculos, como etiquetas continuas para la estimación de características físicas de los obstáculos. Se utilizaron 9 etiquetas en total para reducir la cantidad de neuronas a la salida de la red, y como consecuencia, la cantidad de multiplicaciones y tamaño de la misma.

Etiquetado	Tipo	unidad	comentarios
superficie frontal	Binario	-	Corresponde a cualquier superficie con una inclinación positiva, negativa o nula frente al usuario. Rampas y Suelos
superficie trasera	Binario	-	Corresponde a cualquier superficie con una inclinación positiva, negativa o nula detrás del usuario. Rampas y Suelos
escalón frontal	Binario	-	Corresponde a un peldaño o escalera que se encuentre frente al usuario.
escalón trasero	Binario	-	Corresponde a un peldaño o escalera que se encuentre detrás del usuario.
obstáculo	Binario	-	Corresponde a cualquier superficie con más de $ 15^\circ $ de inclinación, u objeto/-peldaño que supere los 30cm de altura, o directamente una pared o similar.
ángulo superficie frontal	Flotante	$[\circ]$	Ángulo que varía entre -15° y 15° . Escalado entre 0 y 1.
ángulo superficie trasera	Flotante	$[\circ]$	Ángulo que varía entre -15° y 15° . Escalado entre 0 y 1.
altura escalones	Flotante	$[cm]$	Altura que varía entre 1 cm y 30 cm. Escalado entre 0 y 1.
distancia obstáculo	Flotante	$[m]$	Distancia que varía entre 30 cm y 3 m. Escalado entre 0 y 1.

Tabla 5: Etiquetado.

Por otra parte, se pensaron dos tipos de redes, cada una diseñada según los *inputs* que recibían. La red de iteración 1, la cual se puede apreciar en la figura[16], considera tanto los 450 puntos del Lidar interpolados con todas las características descritas en las secciones anteriores. Junto a estas mediciones, se sumaron las del IMU como información adicional a la red, específicamente acelerometro, giroscopio, gravedad (cada uno con sus ejes xyz), roll y pitch.

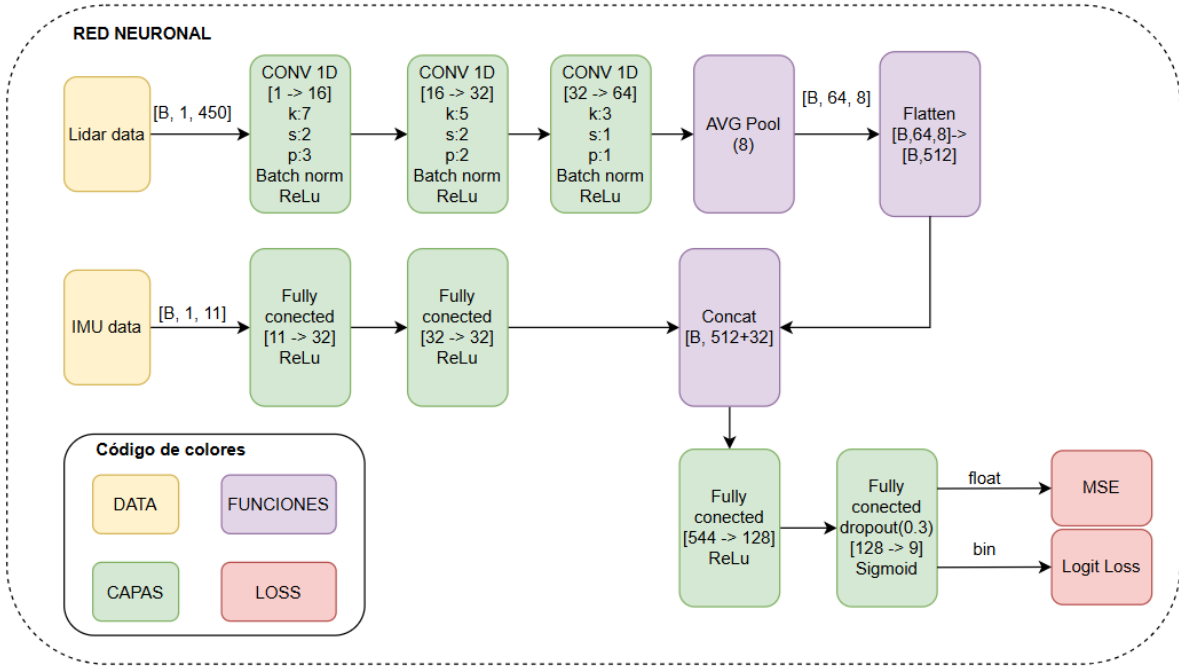


Figura 16: Diagrama red iteración 1

Como se puede apreciar de la figura anterior, los datos del Lidar se pasaron por capas convolucionales, mientras que los del IMU por capas densas. Esto es debido a la correlación espacial que hay en las 450 muestras del Lidar las cuales son aprovechadas de mejor manera con redes convolucionales, mientras que las del IMU no son un array espacial de datos, solo mediciones. También, destacar que se separaron las etiquetas para pasar a la función de error, esto porque, al ser comparadas con etiquetas de distinta naturaleza, tienen que pasar por funciones de error distintas. Específicamente, a través de la librería `Pytorch`, se usaron las funciones de error `BCEWithLogitsLoss` y `MSELoss` para los datos binarios y flotantes respectivamente. Ambas funciones de pérdida se sumaron para ser computadas por la red.

La segunda red que se probó sin los datos del IMU. Se usaron las mismas características de etiquetado descritas y las mismas funciones de error para entrenar la red. En este caso, el input son las 450 muestras del Lidar, interpoladas, pero normalizadas por la el IMU. El diagrama de la red iteración 2 se puede apreciar en la siguiente figura ([17]).

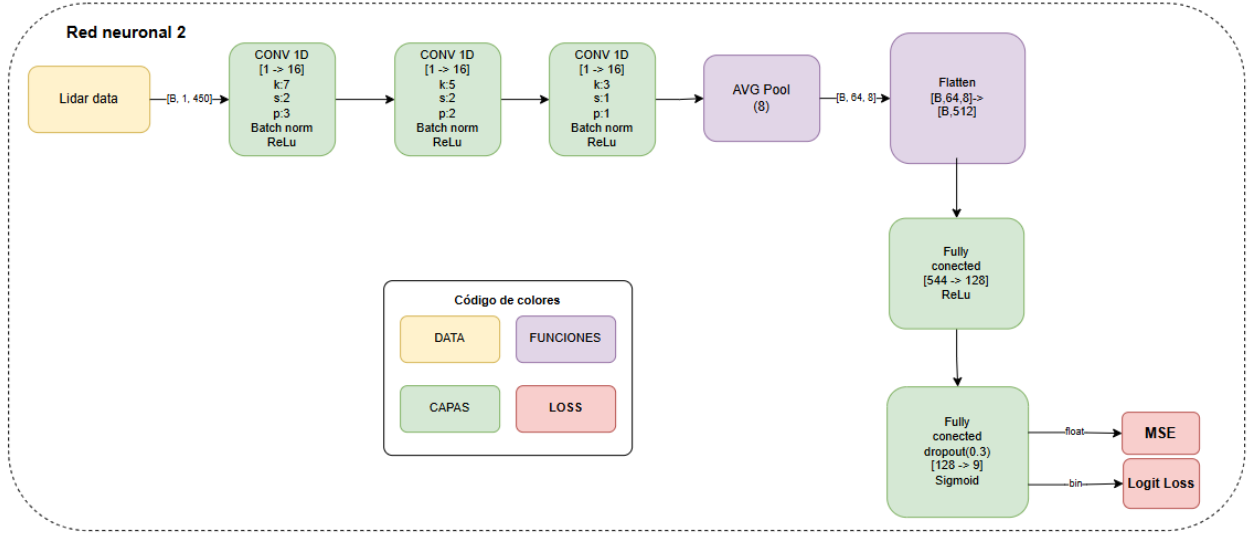


Figura 17: Diagrama red iteración 2

Los resultados de entrenamiento, *Loss*, *Validation Loss*, *Accuracy*, pruebas en tiempo real y limitaciones se presentan en las siguientes secciones.

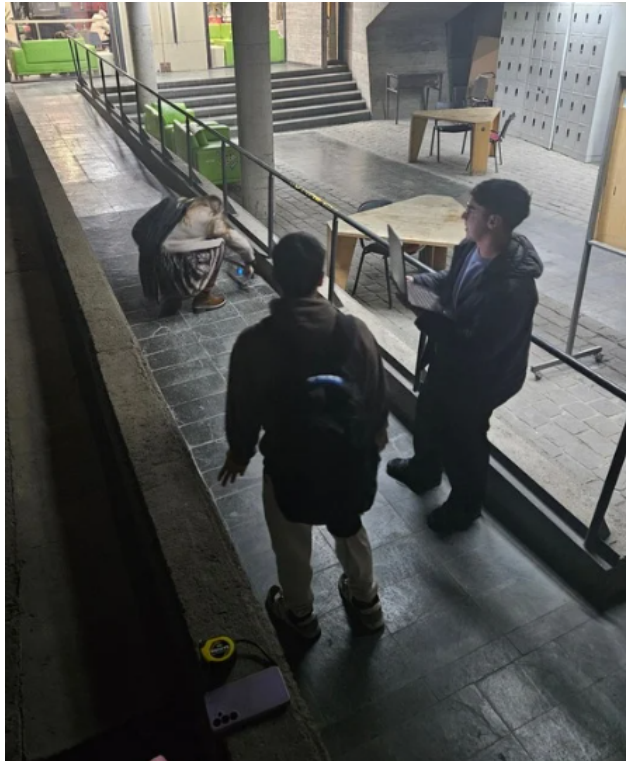
2.3. Adquisición de datos

Para llevar a cabo la adquisición de datos se utilizó un *script* de Python capaz de realizar las capturas y etiquetarlas en el momento, mediante las indicaciones de la persona que mide. Para facilitar y agilizar el proceso de toma de muestras, el código considera una interfaz web local, que permite observar en el momento las mediciones del sensor LIDAR, en sus tres formas (ver figura): medidas crudas, estabilizadas mediante IMU e interpoladas con limitación de máximo para estandarizar las medidas. La interfaz permite conservar las etiquetas del último registro, lo que acelera el proceso y permite seguir los movimientos capturados en tiempo real.

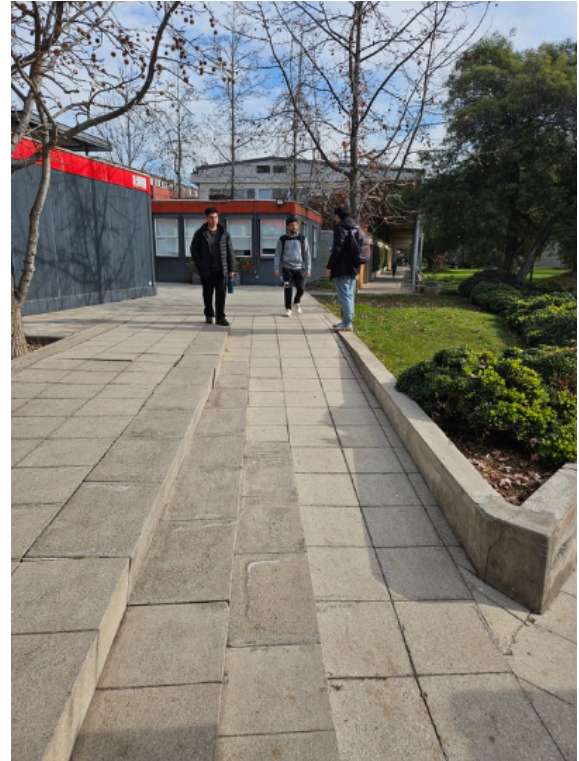


Figura 18: Pagina Local para la Toma de Muestras

Se escogieron más de 50 lugares dentro del campus para servir como puntos de toma de muestras. Siendo todos estos de naturaleza variable, contemplando diversos tipos de suelos, rampas, escaleras y obstáculos. Variando entre distancias, ángulos y tamaños de cada escenario, según corresponda. También se consideraron espacios cerrados y abiertos, con alta y con baja iluminación, exteriores diurnos y nocturnos, con tal de abarcar la mayor cantidad de casos posibles.



(a) Ejemplo de lugar Nocturno e Interior



(b) Ejemplo de lugar Diurno y Exterior

Figura 19: Imágenes de Ejemplo de Adquisición de Datos

Se buscó iterar entre 30 y 60 muestras distintas por cada lugar, consiguiendo así un total de 2630 capturas. Las muestras fueron tomadas de manera estática (análogo a posar para una foto durante unos segundos), variando entre ellas el ángulo del LIDAR con respecto al piso, la distancia al mismo, la distancia a objetos, rapas y escalones. Esto con tal de simular las distintas posiciones del sistema durante una caminata continua. Para el etiquetado se utilizaron como instrumentos de medición una cinta métrica y un nivel digital. El proceso se maximizó en eficiencia cuando, además del sujeto que utilizó el dispositivo de captura, había un encargado de medir distancias y tamaños, otro encargado de medir ángulos, y un último tomando y etiquetando capturas mediante la interfaz. Por lo que el número ideal de integrantes necesarios para el proceso de adquisición de datos es de 4.

2.4. Procesamiento geométrico en tiempo real

Para efectos de entendimiento de esta sección, se utilizará el formato `variable` para referirse a variables del código implementado en la Raspberry.

2.4.1. Filtrado y preprocesamiento de escaneos

Cada vuelta del LiDAR contiene un número variable de puntos, con posibles lecturas inválidas y un espaciado irregular de los ángulos, lo que dificulta entregar una entrada fija para la red neuronal. Para solucionar lo anterior, se implementa una función que estandariza el escaneo a un vector fijo de 450 distancias, correspondientes a ángulos uniformemente espaciados cada 0,8 entre 0 y 359,2.

El proceso que realiza la función en el código que estandariza las mediciones del LiDAR se puede resumir en los siguientes pasos:

1. Se filtran los puntos con distancia igual a cero, que corresponden a retornos inválidos del sensor.
2. Los puntos restantes se ordenan por ángulo.
3. Se aplica interpolación lineal circular sobre los ángulos objetivo.
4. Se identifican espacios faltantes angulares superiores a 5 (secciones con datos faltantes). Los ángulos del vector fijo que caen dentro de los espacios faltantes se les asigna la distancia máxima del sensor (12.000 mm), evitando que la interpolación genere valores en zonas donde el LiDAR no obtuvo datos en la realidad.

2.4.2. Detección y estimación de inclinación de rampas y del plano de suelo

La detección de superficies inclinadas es realizada por la red neuronal a partir de los datos estandarizados y corregidos por IMU del LiDAR. Las salidas binarias `sup_frontal` y `sup_trasera` de la red indican la presencia o no de una superficie frontal o trasera al escaneo, respectivamente. En caso de que efectivamente hayan superficies en el sector frontal o trasero del escaneo, las variables `ang_sup_frontal` y `ang_sup_trasera` estiman el ángulo de inclinación de la superficie en grados, con un rango de cobertura de ± 15 . Cabe destacar que las superficies consideran las rampas (ángulo distinto de 0°) y piso plano (ángulo 0°)

2.4.3. Detección y estimación de altura de escalones

De forma análoga a la detección de rampas (superficies), las salidas binarias `esc_frontal` y `esc_trasera` de la red clasifican la presencia de un escalón de forma frontal o trasera, de acuerdo a lo mencionado. Gracias a esto se puede clasificar si la prótesis se encuentra al inicio de una escalera,

a la mitad de ella o al final. La salida de la red `altura_escalones` estima la altura del escalón en centímetros, con un rango de cobertura de 0 a 30 cm.

2.4.4. Detección de obstáculos frontales en rango 0.3–3.0 m

La salida `obstaculo` de la red clasifica la presencia de un obstáculo al frente del usuario de la prótesis. En caso de que se detecte uno, la salida `dist_obstaculo` entrega una estimación de la distancia en metros, dentro del rango de 0,3 a 3,0 m.

2.5. Validación de latencia extremo a extremo menor o igual a 200 ms

Para medir la latencia de extremo a extremo se usó la librería `Time` y se midió el tiempo en el *while loop*. Naturalmente el Lidar midió a $10Hz \rightarrow 100ms$ porque es la velocidad del mismo para captar el ambiente, mientras que el resto del código, incluyendo la red, tardó 0.023s. Con esto, tenemos que la latencia de extremo a extremo es de 123 ms. Lo cual cumple con el requisito pedido.

```
Tiempo Recolección de Datos: 0.1000 segundos
Tiempo Procesamiento de Datos: 0.0230 segundos
```

Figura 20: Latencia

2.6. Métricas de error y matriz de confusión

Ambas redes descritas en 16 y 17 fueron entrenadas con los datos descritos anteriormente durante 100 épocas. Se dividieron todos los datos considerando 80 % para entrenar, 20 % para validar y testear respectivamente. Los resultados de entrenamiento son los siguientes:

Metrica/Red	Train Loss	Val Loss	Test-Acc
Red iteración 1	0.048	0.0167	99.58 %
Red iteración 2	0.024	0.0247	99.62 %

Tabla 6: Métricas de error para 100 épocas de entrenamiento

Como se puede apreciar, el error de validación más bajo viene de las muestras con IMU incluido. Cuando una red presenta menos Loss con datos de validación con respecto a los que usa para entrenar, significa que fue sobre entrenada, puesto que está consiguiendo un menor error con datos

parecidos entre ellos versus el que consigue con un dato nuevo de entrenamiento. La red de usada en la iteración 2 tuvo mejores resultados de entrenamiento. En general, al usar ambas redes en tiempo real, generaron distintos tipos de problemas, los cuales se van a describir en la siguiente sección, pero la que usó al final fue la de la segunda iteración.

Los valores de *accuracy* de la sección anterior fueron probados con la base de datos separada para test. También se generó la matriz de confusión sobre los datos binarios, es decir, evalúa qué tan bien reconoce un objeto. En general hay mucha combinación de datos binarios los cuales son descritos en la tabla ??, por lo que se decidió por simplicidad, juntar todos los valores que debieron ser cero y el complemento y evaluar en cuántos efectivamente sacó el valor correcto. Los resultados son los siguientes:

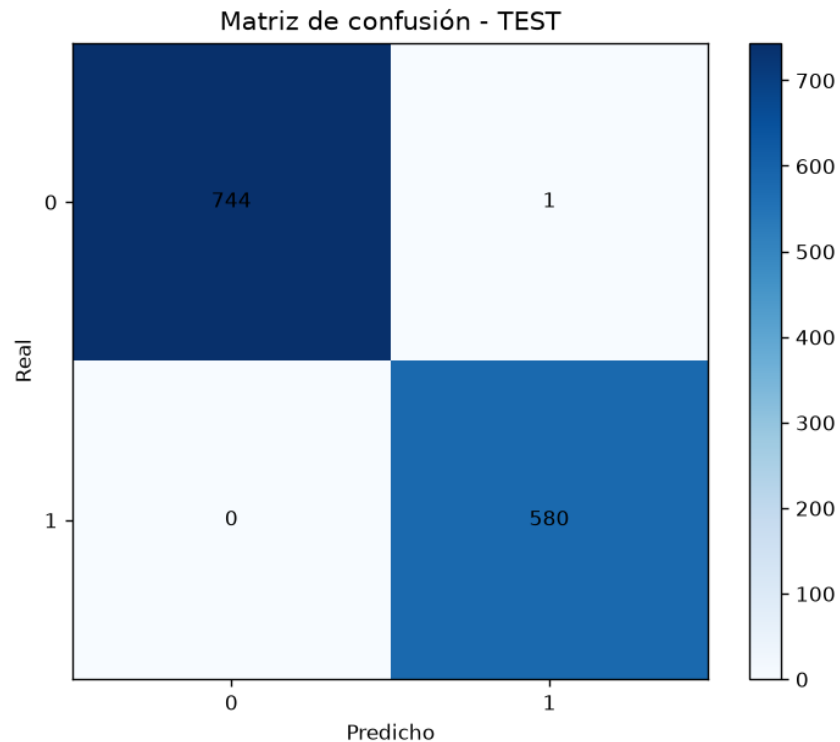


Figura 21: Matriz de confusión sobre Test-dataset para datos binarios

Tal como se espera del *accuracy*, hubo un solo falso uno, con todas estas métricas se esperaba un correcto funcionamiento de la red en tiempo real.

2.7. Validación experimental en entorno controlado con medición de errores geométricos

De la misma manera que se muestra en la figura [19] se probó la red neuronal tanto de la primera, como la segunda iteración. La primera de todas, saturaba todo tipo de objeto y obstáculo en escaleras. Por otra parte, la segunda red, reconocía sin errores los objetos y lograba medir con cierto error el ángulo de las rampas, altura de escalones y distancia a objetos. Sin embargo la red tuvo limitaciones:

- Funciona según lo esperado en entornos abiertos
- En entornos cerrados los obstaculos los reconoce como escalera
- En las rampas tiene un error de $\pm 1.5^\circ$, menor al solicitado de dos grados
- En obstáculos el error es de $\pm 3cm$ para distancias cercanas, menores a 2 metros. A distancias superiores el error aumenta considerablemente
- Es muy sensible a movimiento, si el Lidar se mueve, las predicciones empeoran hasta que se estabilice.

Dentro de todas estas limitantes, las causas son claras. La toma de datos. Si bien, esta fue estricta en el etiquetado, solo se tomaron muestras en entornos abiertos, las muestras no se tomaron en movimiento, puesto que es complejo medir la distancia de forma dinámica para etiquetar esta misma. La falta de movimiento al tomar datos, y no tener muestras en entornos cerrados, hizo que la red solo supiera de entornos muy controlados. Con esto, podemos concluir que sí es posible darle visión a la prótesis, con un tiempo menor a $200ms$, siendo capaz de reconocer el entorno.

Por otra parte, como trabajo futuro, hay que generar una base de datos en movimiento y predecir solo el objeto. La distancia por su parte, es mejor calcularla con los mismos datos del Lidar, con esto delegamos a la red la interpretación de objetos lo cual es de alta complejidad computacional, y esta es muy eficiente en realizar este trabajo. Una vez identificado el objeto, se calculan las distancias matemáticamente para asegurar un error mínimo (el del mismo Lidar) al momento de entregar datos geométricos.

2.8. Repositorio GitHub documentado

Toda la documentación relativa al proyecto, junto con los archivos de código, modelado 3D, entregas y otros se encuentran en el siguiente link a un repositorio público de GitHub.

- Link: <https://github.com/NaBo04/Lidar2D-prediction-on-prosthetic-legs>

2.9. Manual técnico de integración, arquitectura y pruebas

Respecto a la integración física del contenedor con los demás componentes, este debe acoplarse a la carcasa actual de la prótesis, la cual alberga las PCB existentes y la Raspberry Pi 5, fijándose mediante tornillos ubicados en las partes superior e inferior. Asimismo, se debe garantizar que el ángulo cero del LIDAR quede orientado hacia el frente, para asegurar su correcto funcionamiento.

Continuando con las conexiones, se encuentra la alimentación del LIDAR, la cual debe conectarse a una fuente de 12V, ya que este cuenta con regulación interna a 5V. Además, está la conexión USB: en un extremo debe conectarse a la Raspberry Pi 5, mientras que el extremo USB-C debe conectarse al adaptador USB incluido en el contenedor. Por parte del IMU, se debe asegurar la alimentación de 3.3V y su tierra correspondiente, además de las conexiones SDA y SCL, las cuales corresponden a los pines GPIO2 y GPIO3, respectivamente.

Finalmente, respecto a la implementación de software, se deben descargar los archivos directamente desde el repositorio de GitHub e incluirlos en un mismo directorio, el cual debe contener tres archivos de Python, el modelo de la red neuronal y la carpeta correspondiente al *environment*.

Para su uso, se debe abrir la terminal de la Raspberry Pi 5 y, ubicado en el directorio de descarga, activar el *environment* mediante el comando "source env/bin/activate". Esto permitirá cargar las librerías específicas requeridas por el proyecto. A continuación, se debe ejecutar el comando "python script_red_IMU_LIDAR.py" para iniciar el proceso de inferencia.

2.10. Presupuesto final

A continuación, se detallan las compras realizadas para llevar a cabo el proyecto.

Descripción	Precio	Comentarios/Justificación
Banda compresora muslera	\$8.290	Soporte para pruebas: en esta banda se montará el sistema del LiDAR para realizar pruebas previas a la propuesta de instalación del sensor en la prótesis.
Bateria	\$11.000	Soporte para pruebas: se utilizará para alimentar el sistema (5000 mAh).
Velcro Autoadhesivo	\$7.480	Soporte para pruebas: se utilizará para poder remover y ubicar fácilmente la batería en la banda utilizada. Se pegará una banda de velcro a la batería que usaremos para alimentar el sistema con el objetivo de que sea más simple cargarla.
Pack de pernos y tuercas	\$1.490	Para unir las piezas de impresión 3D
Filamento impresión 3D (2 kg)	\$28.000	Soporte para pruebas: se necesita imprimir el soporte donde montaremos el LiDAR y demás componentes del sistema junto al agarre a la banda.
IMU	\$31.878	Implementación: se utilizará para determinar el ángulo de inclinación de la pierna en tiempo real. Comentario: no se encontró un proveedor local confiable y a un precio razonable.
MCI Proto-Pi Shield de Prototipaje	\$3.290	Solicitado por la contraparte para mayor modulación en la aplicación.

Continúa en la siguiente página...

Tabla 7 – Continuación de la página anterior

Descripción	Precio	Comentarios/Justificación
Cable Dupont Hembra Hembra 10cm	\$1.990	Para conectar la IMU con la Raspberry PI.
Inclinómetro nivel digital	\$33.706	Permite aislar la linea de comunicación entre LIDAR 2D y Raspberry Pi 5.
2 Multiplexor	\$8.546	Controlar el uso de Comunicación I2C.
3 Módulo Regulador	\$6.264	Permite reducir voltaje de 5V a 3.3V.
Conversor DC-DC de 12-5 V aislado HiLink 5 W	\$10.546	Alimentación aislada LIDAR 2D.
Módulo aislador digital de doble canal ADUM1201	\$1.083	Para realizar mediciones confiables y precisas de niveles de inclinación para validar las pruebas y resultados.
Huinchas	\$4.390	Para medir la distancia real entre el LIDAR y los obstáculos en la fase de toma de muestras.
Kit 120 Espaciadores	\$6.000	Para espaciar y fijar las PCB del soporte de la prótesis.
Total \$163.953		

Tabla 7: Presupuesto Hito 2.

3. Anexo

Listing 1: Código completo

```
1 # Esto c digo consiste en una estructura de dos procesos
2 # En un nucleo se ejecuta el LiDAR y el IMU, y en otro n cleo se
   procesan los datos, con la
3 # posterior inferencia de la red neuronal. La comunicaci n entre
   procesos se realiza mediante
4 # una cola de multiprocessing.
5
6
7 from servidor_web import lanzar_servidor_web # libreria que contiene
   la funci n para lanzar el servidor web en un hilo separado
8
9 import multiprocessing
10 import threading
11 import time
12 import os
13 import board
14 import busio
15 from pyrpilidar import PyRPlidar
16 from adafruit_bno08x import (
17     BNO_REPORT_ACCELEROMETER,
18     BNO_REPORT_GYROSCOPE,
19     BNO_REPORT_GRAVITY
20 )
21 from adafruit_bno08x.i2c import BNO08X_I2C
22 from collections import deque
23 import numpy as np
24 from LidarRed import LidarCNN
25 import torch
26 import torch.nn as nn
```

```

27
28
29 # =====
30 # CONFIGURACIONES
31 # =====
32
33 NUM_POINTS = 450
34 ANGLE_STEP = 0.8
35 MAX_RANGE_MM = 12000
36 MAX_GAP_DEG = 5
37
38 # ngulos objetivo fijos: 0.0, 0.8, 1.6, ... siempre los mismos.
39 TARGET_ANGLES = np.arange(NUM_POINTS) * ANGLE_STEP
40
41 # =====
42 # FUNCIONES UTILIZADAS
43 # =====
44
45 # Funcion de interpolaci n de vuelta del LiDAR
46 # Recibe una vuelta completa de datos (lista de tuplas) y devuelve
    un vector fijo de 450 distancias.
47 def interpolar_vuelta(vuelta):
48     """Vector fijo de 450 distancias. Interpola con puntos validos;
        huecos
49         grandes (> MAX_GAP_DEG) se asignan a MAX_RANGE_MM (sin retorno /
        lejano)."""
50     angles = np.array([p[0] for p in vuelta], dtype=float)
51     distances = np.array([p[1] for p in vuelta], dtype=float)
52
53     valido = distances > 0.0
54     ang_v = angles[valido]
55     dist_v = distances[valido]

```

```

56
57     if ang_v.size < 2:
58         return np.full(NUM_POINTS, MAX_RANGE_MM)
59
60     orden = np.argsort(ang_v)
61     ang_v = ang_v[orden]
62     dist_v = dist_v[orden]
63
64     dist_interp = np.interp(TARGET_ANGLES, ang_v, dist_v, period
65                             =360)
66
67     # Huecos grandes en la cobertura v lida -> rango m ximo.
68     ext = np.concatenate([ang_v, [ang_v[0] + 360.0]])
69     for i in range(ang_v.size):
70         lo, hi = ext[i], ext[i + 1]
71         if hi - lo > MAX_GAP_DEG:
72             dentro = ((TARGET_ANGLES > lo) & (TARGET_ANGLES < hi)) |
73                     (
74                         (TARGET_ANGLES + 360.0 > lo) & (TARGET_ANGLES +
75                             360.0 < hi)
76                     )
77             dist_interp[dentro] = MAX_RANGE_MM
78
79     return dist_interp
80
81 def compensar_2d(vuelta_completa, imu_capturado):
82     """
83     Desfasa los ngulos crudos del LiDAR utilizando el vector de
84     gravedad del IMU.
85
86     Retorna la lista de puntos rotada y nivelada al horizonte real.
87
88     """
89     # 1. Extraemos la gravedad promediada

```

```

84     grav_x, grav_y, grav_z = imu_capturado['grav_promedio']
85
86     # 2. Calculamos el ngulo actual de la gravedad (en grados)
87     # Estando nivelado: G_x = 0, G_y = -9.8 --> angulo = -90 grados
88     angulo_gravedad_rad = np.arctan2(grav_y, grav_x)
89     angulo_gravedad_deg = np.degrees(angulo_gravedad_rad)
90
91     # 3. Calculamos la desviaci n respecto al ideal (-90)
92     desfase = angulo_gravedad_deg - 90.0
93
94     # 4. Aplicamos el desfase angular a cada tupla
95     vuelta_corregida = []
96     for angulo, distancia, calidad in vuelta_completa:
97         angulo_nuevo = (angulo + desfase) % 360.0
98         vuelta_corregida.append((angulo_nuevo, distancia, calidad))
99
100     return vuelta_corregida
101
102
103 def calcular_roll_pitch(grav_x, grav_y, grav_z):
104     """
105     Calcula roll y pitch (en grados) a partir del vector de gravedad
106     promediado.
107     Convenci n est ndar (aeroespacial), asumiendo el eje Z del
108     sensor "hacia arriba":
109     roll = atan2(grav_y, grav_z)
110     pitch = atan2(-grav_x, sqrt(grav_y^2 + grav_z^2))
111     NOTA: si la orientaci n f sica de montaje del BN0085 es
112     distinta, los signos o
113     los ejes usados aqu pueden necesitar ajuste.
114     """
115     roll_rad = np.arctan2(grav_y, grav_z)

```

```

113     pitch_rad = np.arctan2(-grav_x, np.sqrt(grav_y**2 + grav_z**2))
114     return np.degrees(roll_rad), np.degrees(pitch_rad)
115
116
117 def load_model(device, pesos_path='lidar_cnn_model.pth'):
118     model = LidarCNN().to(device)
119     if os.path.exists(pesos_path):
120         model.load_state_dict(torch.load(pesos_path, map_location=
121                                         device))
122         print(f"Modelo cargado desde: {pesos_path}")
123     else:
124         print(f"No se encontr {pesos_path}'. Se usar un modelo
125               con pesos aleatorios.")
126     model.eval()
127     return model
128
129 def normalizar_datos(X_lidar, X_imu):
130     X_lidar_min = X_lidar.min(dim=1, keepdim=True)[0]
131     X_lidar_max = X_lidar.max(dim=1, keepdim=True)[0]
132     X_lidar_norm = (X_lidar - X_lidar_min) / (X_lidar_max -
133                                             X_lidar_min + 1e-8)
134
135     X_imu_mean = X_imu.mean(dim=1, keepdim=True)
136     X_imu_std = X_imu.std(dim=1, keepdim=True)
137     X_imu_norm = (X_imu - X_imu_mean) / (X_imu_std + 1e-8)
138
139     return X_lidar_norm, X_imu_norm
140
141 def normalizar_lidar(X_lidar):
142     """
143     X_lidar: tensor de forma (B, N) o (N,).

```

```

142     """Normaliza cada fila usando su propio min/max (independiente
    entre filas).
143     """
144     X_lidar = X_lidar.float()
145     if X_lidar.dim() == 1:
146         X_lidar = X_lidar.unsqueeze(0)    # (N,) -> (1, N)
147
148     X_lidar_min = X_lidar.min(dim=1, keepdim=True)[0]
149     X_lidar_max = X_lidar.max(dim=1, keepdim=True)[0]
150     X_lidar_norm = (X_lidar - X_lidar_min) / (X_lidar_max -
        X_lidar_min + 1e-8)
151
152     return X_lidar_norm.float()
153
154 def normalizar_datos(X_imu):
155     X_imu = X_imu.float()
156     if X_imu.dim() == 1:
157         X_imu = X_imu.unsqueeze(0)
158     X_imu_mean = X_imu.mean(dim=1, keepdim=True)[0]
159     X_imu_std = X_imu.std(dim=1, keepdim=True)[0]
160     X_imu_norm = (X_imu - X_imu_mean) / (X_imu_std + 1e-8)
161     return X_imu_norm.float()
162
163
164 # =====
165 # BUFFERS PARA LA MEDIA M VIL
166 # =====
167 # Guardar n exactamente las ltimas 10 muestras
168 historial_grav_x = deque(maxlen=10)
169 historial_grav_y = deque(maxlen=10)
170 historial_grav_z = deque(maxlen=10)
171

```

```

172 # =====
173 # VARIABLE COMPARTIDA (Memoria del Productor)
174 # =====
175 # Inicializamos con valores neutros (gravedad apuntando hacia abajo
    en Y por defecto)
176 # (SE PUEDEN CAMBIAR EL MODO EN EL QUE SE ALMCENA LOS DATOS SI SE
    DESEA)
177 datos_imu_actuales = {
178     'accel': (0.0, 0.0, 0.0),
179     'gyro': (0.0, 0.0, 0.0),
180     'grav_promedio': (0.0, -9.8, 0.0)
181 }
182
183 # =====
184 # HILO SECUNDARIO (Lee el IMU a 100Hz)
185 # =====
186 def lector_imu_hilo():
187     global datos_imu_actuales
188
189     try:
190         # 1. Inicializaci n del bus I2C y el sensor
191         i2c = busio.I2C(board.SCL, board.SDA)
192         bno = BNO08X_I2C(i2c)
193
194         # 2. Habilitaci n de los 3 reportes solicitados
195         bno.enable_feature(BNO_REPORT_ACCELEROMETER)
196         bno.enable_feature(BNO_REPORT_GYROSCOPE)
197         bno.enable_feature(BNO_REPORT_GRAVITY)
198
199         # Pre-llenamos el buffer para que los primeros 100ms no
            promedien con ceros
200         for _ in range(10):

```

```

201         historial_grav_x.append(0.0)
202         historial_grav_y.append(-9.8)
203         historial_grav_z.append(0.0)
204
205         print("[IMU]_BN0085_inicializado._Leyendo_Accel,_Gyro_y_
           Gravedad_a_100Hz.")
206
207     except Exception as e:
208         print(f"[ERROR_IMU]_No_se_pudo_inicializar_el_sensor._Revisa
           _I2C._Detalle:{e}")
209         return # Termina el hilo de forma segura si no hay hardware
210
211 # 3. Bucle infinito de lectura
212 while True:
213     try:
214         # Extraemos las tuplas (x, y, z) directamente del
           hardware
215         accel_x, accel_y, accel_z = bno.acceleration
216         gyro_x, gyro_y, gyro_z = bno.gyro
217         grav_x, grav_y, grav_z = bno.gravity
218
219         # Agregamos la nueva lectura al historial (empuja y
           borra la m s vieja)
220         historial_grav_x.append(grav_x)
221         historial_grav_y.append(grav_y)
222         historial_grav_z.append(grav_z)
223
224         # Calculamos el promedio de los ltimos 10 datos (
           aplasta el ruido)
225         prom_x = sum(historial_grav_x) / 10.0
226         prom_y = sum(historial_grav_y) / 10.0
227         prom_z = sum(historial_grav_z) / 10.0

```

```

228
229     # Actualizamos el DICCIONARIO GLOBAL (SE PUEDEN CAMBIAR
        EL MODO EN EL QUE SE ALMCENA LOS DATOS SI SE DESEA)
230     datos_imu_actuales = {
231         'accel': (accel_x, accel_y, accel_z),
232         'gyro': (gyro_x, gyro_y, gyro_z),
233         'grav_promedio': (prom_x, prom_y, prom_z)
234     }
235
236     # Retardo exacto para mantener la frecuencia de 100Hz
        (10ms)
237     time.sleep(0.01)
238
239     except OSError:
240         # Los buses I2C a veces tienen micro-desconexiones
            f s i c a s .
241         # Capturamos el error silenciosamente para que el hilo
            no muera
242         # y reintente en el siguiente ciclo.
243         pass
244
245 # =====
246 # PROCESO 1: SUPER PRODUCTOR (Anclado al N cleo 1)
247 # =====
248 def productor_sensores(cola_datos):
249     os.sched_setaffinity(0, {1})
250
251     # Arrancamos el Hilo del IMU en el fondo
252     hilo_imu = threading.Thread(target=lector_imu_hilo, daemon=True)
253     hilo_imu.start()
254
255     # Inicializamos el LiDAR

```

```

256     lidar = PyRPlidar()
257     lidar.connect(port="/dev/ttyUSB0", baudrate=460800, timeout=3)
258
259     # LIMPIEZA DE BUFFER (Crucial para reiniciar sin errores)
260     lidar.stop()
261     time.sleep(0.2)
262     lidar.lidar_serial._serial.reset_input_buffer()
263     lidar.lidar_serial._serial.reset_output_buffer()
264     time.sleep(0.3)
265
266     lidar.set_motor_pwm(500)
267     time.sleep(2) # Esperamos a que el motor alcance velocidad
        estable
268
269     scan_generator_func = lidar.start_scan()
270
271     # OPTIMIZACIN SUPREMA!: Instanciamos el iterador de flujo
        cont nuo una sola vez afuera
272     generador = scan_generator_func()
273
274     # Enganchamos el flujo buscando el PRIMER inicio oficial antes
        de entrar al while
275     proximo_inicio = None
276     for scan in generador:
277         if scan.start_flag:
278             proximo_inicio = scan
279             break
280
281     print("[SISTEMA] Iniciando recolecci n sincronizada LiDAR + IMU
        (Inline Stream)...")
282

```

```

283     # 3. Bucle principal unificado (Todo en una sola estructura
        lineal)
284     while True:
285         try:
286             # Inicializamos la lista directamente con el punto que
                cerr la vuelta anterior
287             # As no perdemos muestras y mantenemos el desfase de
                hardware controlado
288             vuelta_completa = [(proximo_inicio.angle, proximo_inicio
                .distance, proximo_inicio.quality)]
289
290             # RECOLECCION PURA EN UN SOLO BUCLE EN LINEA
291             for scan in generador:
292                 if scan.start_flag:
293                     # Capturamos el inicio de la SIGUIENTE vuelta y
                        rompemos el bucle actual
294                     proximo_inicio = scan
295                     break
296                     # Guardamos los puntos intermedios a velocidad
                        nativa de C
297                     vuelta_completa.append((scan.angle, scan.distance,
                        scan.quality))
298
299                     # Ordenamos los puntos por angulo (vital para
                        interpolar_vuelta en el Ncleo 2)
300                     vuelta_completa.sort(key=lambda p: p[0])
301
302                     # Capturamos la foto exacta del IMU promediado en este
                        instante de cierre
303                     imu_capturado = datos_imu_actuales.copy()
304
305                     # Enviamos el paquete combinado al Cerebro

```

```

306         paquete = (vuelta_completa, imu_capturado)
307         cola_datos.put_nowait(paquete)
308
309     except multiprocessing.queues.Full:
310         # Si el Cerebro se satura, saltamos el paquete para
311         # proteger los tiempos reales
312         pass
313     except KeyboardInterrupt:
314         break
315
316     # Apagado seguro
317     lidar.stop()
318     lidar.set_motor_pwm(0)
319     lidar.disconnect()
320
321 # =====
322 # PROCESO 2: EL CEREBRO (Anclado al Nucleo 2)
323 # =====
324 def consumidor_datos(cola_datos, cola_web):
325     os.sched_setaffinity(0, {2})
326
327     # cargar modelo (antes del while)
328     modelo = load_model(device='cpu', pesos_path='lidar_cnn_model.
329     pth')
330     contador = 0
331
332     while True:
333         # Esperamos y desempaquetamos los datos
334         paquete = cola_datos.get()
335         vuelta_completa, imu_capturado = paquete

```

```

336     # NIVELACION: Corregimos los ángulos con la gravedad del
           IMU
337     vuelta_nivelada = compensar_2d(vuelta_completa,
           imu_capturado)
338
339     # INTERPOLACION: Forzamos los puntos al vector fijo de 450
           elementos
340     # - vuelta_interpolada_cruda: LiDAR crudo (sin nivelar
           por IMU), interpolado
341     # - vuelta_interpolada_nivelada: LiDAR nivelado por IMU,
           interpolado
342     vuelta_interpolada_cruda = interpolar_vuelta(
           vuelta_completa)
343     vuelta_interpolada_nivelada = interpolar_vuelta(
           vuelta_nivelada)
344
345     # ROLL / PITCH: derivados del vector de gravedad promediado
346     accel_x, accel_y, accel_z = imu_capturado['accel']
347     grav_x, grav_y, grav_z = imu_capturado['grav_promedio']
348     roll_deg, pitch_deg = calcular_roll_pitch(grav_x, grav_y,
           grav_z)
349
350     # La inferencia recibe un vector de 450 distancias crudos
           interpolados mas 8 datos del imu (sin giroscopio) y
           devuelve un diccionario con la predicción
351     X_lidar = torch.from_numpy(vuelta_interpolada_nivelada.
           astype('float32')).unsqueeze(0) # (1, 450)
352     X_imu = torch.from_numpy(np.array([
353         accel_x, accel_y, accel_z, grav_x, grav_y, grav_z,
           roll_deg, pitch_deg
354     ], dtype=np.float32)).unsqueeze(0) # (1, 8)
355

```

```

356     # Normalizamos los datos de entrada
357     X_lidar_norm = normalizar_lidar(X_lidar)
358     X_imu_norm = normalizar_datos(X_imu)
359
360     # Inferencia con el modelo
361     with torch.no_grad():
362         salida = modelo(X_lidar_norm).squeeze(0)
363         prediccion = salida.cpu().numpy() # Convertimos a numpy
            para enviar al web [9 ETIQUETAS {sup_frontal,
            sup_trasera, esc_frontal, esc_trasera, obstaculo,
            ang_sup_frontal, ang_sup_trasera, altura_escalones,
            dist_obstaculo}]
364
365     prob_binarias = torch.sigmoid(salida[:5]).cpu().numpy()
366     valores_regresion = torch.sigmoid(salida[5:]).cpu().numpy()
367     pred_binarias = (prob_binarias > 0.5).astype(int)
368
369     nombres_etiquetas = [
370         "sup_frontal_(bin)", "sup_trasera_(bin)", "esc_frontal_(
            bin)",
371         "esc_trasera_(bin)", "obstaculo_(bin)", "ang_sup_frontal
            _(float)",
372         "ang_sup_trasera_(float)", "altura_escalones_(float)", "
            dist_obstaculo_(float)"
373     ]
374
375     if contador == 15:
376         print(f"\n===_INFERENCIA_RANDOM_EN_VALIDACION_===")
377         for i, nombre in enumerate(nombres_etiquetas[:5]):
378             print(f"{nombre}:_pred={pred_binarias[i]}_(prob={
                prob_binarias[i]:.4f})")

```

```

379         for i, nombre in enumerate(nombres_etiquetas[5:], start
380                                     =5):
381             if nombre == "ang_sup_frontal_(float)" or nombre ==
382                 "ang_sup_trasera_(float)":
383                 print(f"{nombre}:_pred={{(valores_regresion[i
384                     -5]*30-15):.4f}}")
385             if nombre == "dist_obstaculo_(float)":
386                 print(f"{nombre}:_pred={{(valores_regresion[i
387                     -5]*2.7+0.3):.4f}}")
388             if nombre == "altura_escalones_(float)":
389                 print(f"{nombre}:_pred={{(valores_regresion[i
390                     -5]*30):.4f}}")
391             print("=====\n")
392             contador = 0
393
394     contador += 1
395
396     # 4. EXPORTACION WEB (Todo el paquete en un diccionario de
397     # una linea)
398     try:
399         cola_web.put_nowait({
400             "cruda": vuelta_completa,
401             "nivelada": vuelta_nivelada,
402             "interpolada_cruda": vuelta_interpolada_cruda.tolist
403                 (),
404             "interpolada": vuelta_interpolada_nivelada.tolist(),
405             "imu": imu_capturado,
406             "imu_roll_deg": roll_deg,
407             "imu_pitch_deg": pitch_deg
408         })
409     except multiprocessing.queues.Full: pass
410
411

```

```

404
405
406 # =====
407 # ARRANQUE PRINCIPAL DEL PROGRAMA
408 # =====
409 if __name__ == '__main__':
410     # 1. Definici n de las colas
411     cola_compartida = multiprocessing.Queue(maxsize=3)
412     cola_web = multiprocessing.Queue(maxsize=1)
413
414     # 2. Creaci n de los procesos
415     proceso_web = multiprocessing.Process(target=lanzar_servidor_web
416     , args=(cola_web,))
417
418     proceso_sensores = multiprocessing.Process(target=
419     productor_sensores , args=(cola_compartida,))
420
421     # Se inyectan AMBAS colas al cerebro
422     proceso_cerebro = multiprocessing.Process(target=
423     consumidor_datos , args=(cola_compartida, cola_web))
424
425     try:
426
427         # 3. Arranque de los procesos
428         proceso_web.start()
429         proceso_cerebro.start()
430         proceso_sensores.start()
431
432         # 4. Mantener vivo el script principal
433         proceso_sensores.join()
434         proceso_cerebro.join()
435         proceso_web.join()
436
437     except KeyboardInterrupt:

```

```
433     print("\n[SISTEMA] Deteniendo procesos de forma segura...")
434     proceso_sensores.terminate()
435     proceso_cerebro.terminate()
436     proceso_web.terminate()
```